

Desempenho de Aplicações Web Python/Django Containerizadas com Docker em Ambientes de Alta Concorrência

Pedro Lima de Assunção Filho¹, Prof. Dr. Mário Antonio Meireles Teixeira²

¹Departamento de Informática – Universidade Federal do Maranhão (UFMA)
Av. dos Portugueses, 1966 – Cidade Universitária Dom Delgado – 65080-805 – São Luís – MA – Brazil

pedro.assuncao@email.com, mario.teixeira@ufma.br

Abstract. *This paper presents a methodology for evaluating the performance of web applications developed with Python/Django and deployed in containerized environments using Docker. To this end, the experiments were carried out in an automated and replicable setup designed to manage multiple container instances in a controlled manner through Docker Compose. Each instance comprised a Django backend configured with Gunicorn and Uvicorn (five fixed workers), along with a PostgreSQL database, ensuring consistency and isolation during execution.*

Based on this setup, several load scenarios were designed and executed, varying the number of virtual users and the type of REST operation (GET, POST, and DELETE) to analyze the system's behavior under different concurrency levels. The metrics evaluated included average response time, requests per second (RPS), failure rate, and computational resource usage.

In this context, the creation of the automated environment enabled standardized data collection and ensured the reproducibility of experiments, representing a relevant methodological contribution to performance studies in containerized systems. Consequently, the results provided an initial overview of the scalability and stability of the analyzed architecture, serving as a foundation for future optimizations and investigations into the impact of concurrency on Python/Django applications.

Resumo. *Este trabalho apresenta uma metodologia para a avaliação de desempenho de aplicações web desenvolvidas em Python/Django e executadas em ambientes containerizados com Docker. Para isso, os experimentos foram realizados em um ambiente automatizado e replicável, projetado para gerenciar, de forma controlada, múltiplas instâncias de containers por meio do Docker Compose. Cada instância foi composta por um backend Django, configurado com Gunicorn e Uvicorn (cinco workers fixos), além de um banco de dados PostgreSQL, garantindo consistência e isolamento durante as execuções.*

Com base nessa estrutura, foram elaborados e executados diversos cenários de carga, variando o número de usuários virtuais e o tipo de operação REST (GET, POST e DELETE), a fim de observar o comportamento do sistema sob diferentes níveis de concorrência. As métricas analisadas incluíram tempo médio de resposta, requisições por segundo (RPS), taxa de falhas e consumo de recursos computacionais.

Nesse contexto, a criação do ambiente automatizado permitiu padronizar a coleta de dados e assegurar a reprodutibilidade dos experimentos, configurando-se como uma contribuição metodológica relevante para estudos de desempenho em sistemas containerizados. Assim, os resultados obtidos ofereceram uma visão inicial sobre a escalabilidade e a estabilidade da arquitetura analisada, servindo como ponto de partida para futuras otimizações e investigações acerca do impacto da concorrência em aplicações Python/Django.

1. Introdução

Aplicações web modernas demandam escalabilidade, portabilidade e isolamento entre ambientes de execução. Nesse contexto, a containerização se consolidou como uma abordagem essencial para a implantação de sistemas distribuídos, permitindo a replicação fidedigna de ambientes e facilitando estratégias de escalonamento horizontal [Docker Inc. 2025].

Nesse cenário, o Docker destaca-se como a principal tecnologia de containers, enquanto frameworks como o Django oferecem uma estrutura robusta para o desenvolvimento web em Python [Django Software Foundation 2025]. Essa combinação foi escolhida por sua popularidade, maturidade e facilidade de replicação em estudos experimentais. Além disso, empregaram-se o Gunicorn como servidor WSGI e o Uvicorn como worker ASGI, o que assegura compatibilidade com aplicações síncronas e assíncronas, aproximando o ambiente de teste das condições reais de produção. Entretanto, apesar da ampla adoção, o estudo sistemático de desempenho dessa combinação tecnológica em cenários de alta concorrência ainda carece de aprofundamento experimental.

Dessa forma, este trabalho investiga como diferentes níveis de carga impactam o desempenho de APIs desenvolvidas em Python/Django quando executadas em containers Docker, buscando identificar limites operacionais e gargalos de arquitetura. Assim, a principal contribuição deste estudo consiste na construção de um ambiente experimental replicável, voltado à análise quantitativa do desempenho de aplicações containerizadas sob diferentes condições de carga.

2. Trabalhos Relacionados

O uso de containers tem se tornado prática consolidada na engenharia de software devido aos ganhos de portabilidade, isolamento e automação de implantação. Trabalhos como [IEEE 2021] demonstram o papel do Docker no provisionamento automatizado e na otimização de pipelines de deploy.

No contexto de desempenho de frameworks web, há evidências de que o Node.js se beneficia do modelo orientado a eventos e de I/O não bloqueante em cenários de alta concorrência, o que favorece o throughput em serviços web [Iyer 2015, Bueno 2023]. De forma análoga, em Java, estudos recentes mostram ganhos relevantes com Virtual Threads (Projeto Loom) em cargas de alto volume, sobretudo em workloads I/O-bound, com melhorias de throughput e latência em relação a threads convencionais [Pandita 2024]. Contudo, tecnologias baseadas em Python permanecem amplamente adotadas devido à produtividade e flexibilidade, reforçando a necessidade de investigações adicionais sobre uso eficiente de recursos em ambientes containerizados.

Assim, este estudo diferencia-se ao avaliar especificamente o conjunto Python/Django + Docker + Gunicorn/Uvicorn, com monitoramento sistemático e métricas detalhadas sob diferentes níveis de exigência operacional — um recorte ainda pouco explorado na literatura nacional [Gunicorn Project 2025, Uvicorn Project 2025, Grafana Labs 2025].

3. Metodologia

Os experimentos foram conduzidos em um ambiente controlado que visou avaliar o comportamento de uma aplicação Django containerizada sob diferentes níveis de concorrência. A aplicação consistiu em uma API REST implementada em Django e empacotada em uma imagem Docker baseada em Python Slim. O backend utilizou Gunicorn aliado ao Uvicorn como workers ASGI (cinco fixos), enquanto o banco de dados PostgreSQL foi orquestrado conjuntamente via Docker Compose, assegurando isolamento e reprodutibilidade.

- Os testes ocorreram em dois ambientes: Servidor em nuvem (2 vCPUs, 4 GB RAM, Ubuntu 24.10 x64) e máquina local (AMD Ryzen 7 3700U, 8 GB RAM).

O K6 foi empregado para geração das cargas e o Grafana, acoplado ao Prometheus, para monitoramento do consumo de CPU e memória. Foram testadas quatro operações REST representativas: GET (leitura), POST (escrita), DELETE (exclusão) e GET/POST (misto), sob cargas de 50, 100, 250 e 500 usuários virtuais (VUs), totalizando 192 execuções com duração entre 6 e 10 minutos.

As métricas coletadas abrangeram o tempo médio de resposta (ms), as requisições por segundo (RPS), o percentual de falhas e a utilização de recursos; em seguida, os dados foram agregados e comparados entre os diferentes cenários, procedimento que possibilitou mensurar de forma sistemática a resiliência e o comportamento do sistema sob pressões progressivas de concorrência, refletindo situações típicas de uso real em aplicações web containerizadas.

4. Resultados e Discussão

Os experimentos realizados permitiram observar, de forma sistemática, o comportamento de uma aplicação Python/Django em ambiente containerizado sob diferentes condições de carga. Foram conduzidas 192 execuções, abrangendo múltiplas combinações de CPU, memória e níveis de concorrência (50, 100, 250 e 500 usuários virtuais), totalizando aproximadamente 26 horas de testes contínuos. O monitoramento foi realizado com o Grafana, enquanto o K6 automatizou a geração das cargas e o registro das métricas, garantindo padronização e reprodutibilidade dos experimentos.

Além disso, a execução dos testes foi coordenada por um sistema automatizado desenvolvido no âmbito do projeto — o Docker-Manager —, uma aplicação própria criada para orquestrar e replicar automaticamente instâncias de containers compostos pelo back-end (Django, Gunicorn e Uvicorn) e banco de dados PostgreSQL. Dessa forma, tornou-se possível conduzir os experimentos de maneira controlada e repetitiva, assegurando consistência entre os cenários de execução e contribuindo para a criação de um ambiente de testes totalmente replicável.

De modo geral, o sistema apresentou estabilidade satisfatória até 100 usuários simultâneos, mantendo baixas latências e ausência de falhas perceptíveis. A partir desse ponto, contudo, observou-se uma degradação progressiva do desempenho, caracterizada pela redução do número de requisições por segundo (RPS) e pelo aumento do tempo médio de resposta. Esse comportamento evidencia o início da saturação da CPU, identificada como o principal fator limitante da arquitetura, enquanto o consumo de memória manteve-se estável em todos os cenários analisados.

A análise por tipo de operação revelou diferenças expressivas entre os endpoints testados. O método GET demonstrou ser o mais eficiente e estável, apresentando menores tempos de resposta e maior RPS. Por outro lado, o POST, que envolve escrita no banco de dados, exibiu latências superiores, sugerindo gargalos associados ao PostgreSQL em situações de concorrência elevada. Já o DELETE mostrou-se o ponto mais sensível do sistema, acumulando até 50% de falhas em cenários acima de 100 usuários, o que indica limitações no tratamento de exclusões simultâneas. Por fim, os testes mistos (GET/POST) apresentaram comportamento intermediário, com taxas de falhas residuais inferiores a 0,05%, reforçando a consistência da arquitetura em operações combinadas.

Dessa forma, os resultados reforçam que o processamento da CPU é o componente mais crítico em cenários de alta demanda, uma vez que a redução do RPS esteve diretamente relacionada ao aumento da taxa de falhas. Assim, a faixa de 50 a 100 usuários simultâneos pode ser considerada a zona de estabilidade operacional, oferecendo o melhor equilíbrio entre desempenho, confiabilidade e uso de recursos. Nesse intervalo, a arquitetura manteve desempenho consistente, demonstrando capacidade adequada para ambientes de produção de pequeno e médio porte.

Adicionalmente, a utilização de um ambiente automatizado e replicável, apoiado pelo Docker-Manager, configurou-se como uma contribuição metodológica significativa. A integração entre Docker, Grafana e K6 permitiu a execução coordenada dos testes e o monitoramento contínuo dos recursos, o que reduziu interferências externas e assegurou comparabilidade entre os diferentes cenários experimentais. Essa abordagem estabelece uma base sólida para futuras investigações voltadas à análise de desempenho e escalabilidade em sistemas containerizados.

Por fim, os resultados obtidos alinham-se com estudos anteriores que apontam o impacto da CPU na estabilidade de aplicações baseadas em Django e PostgreSQL sob alta concorrência. Entretanto, este trabalho diferencia-se ao evidenciar, de forma empírica, o comportamento distinto entre operações REST e os limites de escalabilidade específicos dessa arquitetura. As Figuras 1 a 4 apresentam a evolução da latência média, o comportamento do RPS e o uso de recursos sob diferentes níveis de carga, complementando visualmente a discussão apresentada.

4.1. Tabelas e Gráficos

Table 1. (resumo) – Métricas coletadas em diferentes cenários de carga:

CPU	Memória	Backend	SGBD	Operações	VUs	Tempo Resp	Num. Req.	Num. Erros	Util. CPU Serv.	Mem. (bytes)	RPS	% Falhas
1 CPU	1 GB	Django	Postgres	GET	50	370.03	12216	0	214.91	324372070.4	76.35	0.00
1 CPU	1 GB	Django	Postgres	GET	250	2592.79	17454	0	251.56	345165312	109.09	0.00
1 CPU	1 GB	Django	Postgres	GET	500	2954.83	16546	562	279.09	375100114.82	103.41	3.40
1 CPU	1 GB	Django	Postgres	POST	50	237.40	7284	0	149.18	321438326.15	45.53	0.00
1 CPU	1 GB	Django	Postgres	POST	250	7386.27	5483	1277	268.55	441251840	34.27	23.29
1 CPU	1 GB	Django	Postgres	POST	500	27774.70	3330	1566	285.16	450354944	20.81	47.03
1 CPU	1 GB	Django	Postgres	GET/POST	50	783.19	5798	0	227.56	441207886.76	36.24	0.00
1 CPU	1 GB	Django	Postgres	GET/POST	250	3992.11	5940	324	279.81	587870208	37.13	5.45
1 CPU	1 GB	Django	Postgres	GET/POST	500	9159.96	5404	1189	280.81	587895076.57	33.78	22.00
1 CPU	1 GB	Django	Postgres	DELETE	50	439.46	10256	4919	321.49	324125900.8	64.10	47.96
1 CPU	1 GB	Django	Postgres	DELETE	250	2209.66	10422	4788	387.64	334213360.94	65.14	45.94
1 CPU	1 GB	Django	Postgres	DELETE	500	2955.27	8910	4249	416.39	345950810.35	55.69	47.69
1 CPU	2 GB	Django	Postgres	GET	50	368.80	12286	0	223.34	312991488	298.49	0.00
1 CPU	2 GB	Django	Postgres	GET	250	1325.56	17324	0	248.71	331922944	316.55	0.00
1 CPU	2 GB	Django	Postgres	GET	500	2995.32	16238	629	285.76	380620077.17	362.99	3.87
1 CPU	2 GB	Django	Postgres	POST	50	230.20	7332	0	160.94	314381750.85	299.82	0.00
1 CPU	2 GB	Django	Postgres	POST	250	6987.72	5717	1157	259.87	363960905.14	347.10	20.24
1 CPU	2 GB	Django	Postgres	POST	500	25698.40	3513	1645	275.60	456676966.4	435.52	46.83
1 CPU	2 GB	Django	Postgres	GET/POST	50	789.01	5777	0	231.04	448645907.69	427.86	0.00
1 CPU	2 GB	Django	Postgres	GET/POST	250	4159.47	5773	300	275.09	583757824	556.71	5.20
1 CPU	2 GB	Django	Postgres	GET/POST	500	8839.68	5716	1255	295.37	629262628.57	600.11	21.96
1 CPU	2 GB	Django	Postgres	DELETE	50	446.22	10091	4800	326.33	317088668.9	302.40	47.57
1 CPU	2 GB	Django	Postgres	DELETE	250	2094.31	10955	5089	367.94	342880876.6	326.99	46.45
1 CPU	2 GB	Django	Postgres	DELETE	500	6453.33	7559	3358	410.77	351017022.06	334.76	44.42
2 CPUs	1 GB	Django	Postgres	GET	50	370.20	12214	0	215.69	320656179.2	305.80	0.00
2 CPUs	1 GB	Django	Postgres	GET	250	1390.89	16542	0	251.77	340171008	324.41	0.00
2 CPUs	1 GB	Django	Postgres	GET	500	3065.47	16212	566	295.42	392937953.88	374.73	3.49
2 CPUs	1 GB	Django	Postgres	POST	50	234.74	7303	0	155.54	321082289.23	306.21	0.00
2 CPUs	1 GB	Django	Postgres	POST	250	6428.78	6142	1169	250.30	446081496.61	425.42	19.03
2 CPUs	1 GB	Django	Postgres	POST	500	24622.91	3616	1401	266.72	439300642.13	418.95	38.74
2 CPUs	1 GB	Django	Postgres	GET/POST	50	781.03	5819	0	225.94	443684706.46	423.13	0.00
2 CPUs	1 GB	Django	Postgres	GET/POST	250	4106.07	5810	281	269.82	566047305.14	539.82	4.84
2 CPUs	1 GB	Django	Postgres	GET/POST	500	9501.77	5309	0	301.27	607235003.73	579.10	0.00
2 CPUs	1 GB	Django	Postgres	DELETE	50	467.84	9626	4374	332.84	317320291.09	302.62	45.44
2 CPUs	1 GB	Django	Postgres	DELETE	250	2102.87	11135	5837	371.29	342064655.51	326.22	52.42
2 CPUs	1 GB	Django	Postgres	DELETE	500	6780.40	7	3260	429.02	338463563.29	322.78	44.25
2 CPUs	2 GB	Django	Postgres	GET	50	341.46	13250	0	218.93	315757056	301.13	0.00
2 CPUs	2 GB	Django	Postgres	GET	250	1371.07	16862	0	250.01	329498368	314.23	0.00
2 CPUs	2 GB	Django	Postgres	GET	500	2955.37	16648	549	289.30	377108118.58	359.64	3.30
2 CPUs	2 GB	Django	Postgres	POST	50	233.33	7313	0	159.44	314656768	300.08	0.00
2 CPUs	2 GB	Django	Postgres	POST	250	6364.90	6178	1133	244.73	440007443.69	419.62	18.34
2 CPUs	2 GB	Django	Postgres	POST	500	24247.83	3737	1570	267.84	419826619.73	400.38	42.01
2 CPUs	2 GB	Django	Postgres	GET/POST	50	768.26	5902	2	223.76	445002673.23	424.39	0.03
2 CPUs	2 GB	Django	Postgres	GET/POST	250	3949.45	5973	297	280.88	607602980.57	579.46	4.97
2 CPUs	2 GB	Django	Postgres	GET/POST	500	9266.32	5540	1275	290.25	605487104	577.44	23.01
2 CPUs	2 GB	Django	Postgres	DELETE	50	424.45	10611	5482	307.43	318589064.53	303.83	51.66
2 CPUs	2 GB	Django	Postgres	DELETE	250	2115.59	11042	5258	382.91	349235137.93	333.06	47.62
2 CPUs	2 GB	Django	Postgres	DELETE	500	6353.25	7648	3428	435.82	372335796.7	355.09	44.82

Table 2. Métricas coletadas em cenários com 100 usuários simultâneos

CPU	Memória	Backend	SGBD	Operações	VUs	Tempo Resp	Num. Req.	Num. Erros	Util. CPU Serv.	Mem. (bytes)	Mem. (MB)	RPS	% Falhas
1 CPU	1 GB	Django	Postgres	GET	100	625.88	14530	0	230.34	335164757.33	319.64	90.81	0.00
1 CPU	1 GB	Django	Postgres	POST	100	548.29	11659	0	208.56	328669971.69	313.44	72.87	0.00
1 CPU	1 GB	Django	Postgres	GET/POST	100	1548.07	5957	0	256.69	508626156.30	485.06	37.23	0.00
1 CPU	1 GB	Django	Postgres	DELETE	100	834.12	10840	5361	339.73	323317760.00	308.34	67.75	49.46
1 CPU	2 GB	Django	Postgres	GET	100	612.20	14868	0	238.32	321483264.00	306.59	92.93	0.00
1 CPU	2 GB	Django	Postgres	POST	100	684.10	10727	0	210.93	337077169.23	321.46	67.04	0.00
1 CPU	2 GB	Django	Postgres	GET/POST	100	1584.99	5778	1	253.06	477261824.00	455.15	36.11	0.02
1 CPU	2 GB	Django	Postgres	DELETE	100	844.51	10724	5211	344.35	321811294.31	306.90	67.02	48.59
2 CPUs	1 GB	Django	Postgres	GET	100	610.62	14912	0	231.97	330578602.66	315.26	93.20	0.00
2 CPUs	1 GB	Django	Postgres	POST	100	600.22	11277	0	210.35	325562998.15	310.48	70.48	0.00
2 CPUs	1 GB	Django	Postgres	GET/POST	100	1512.86	6062	1	249.18	511525494.15	487.83	37.89	0.02
2 CPUs	1 GB	Django	Postgres	DELETE	100	857.60	10538	5108	336.70	333424806.05	317.98	65.86	48.47
2 CPUs	2 GB	Django	Postgres	GET	100	594.70	15282	0	237.38	317267456.00	302.57	95.51	0.00
2 CPUs	2 GB	Django	Postgres	POST	100	669.76	10810	0	218.63	324649252.57	309.61	67.56	0.00
2 CPUs	2 GB	Django	Postgres	GET/POST	100	1511.20	6082	1	253.91	520545516.30	496.43	38.01	0.02
2 CPUs	2 GB	Django	Postgres	DELETE	100	857.11	10530	5045	342.75	329366042.94	314.11	65.81	47.91

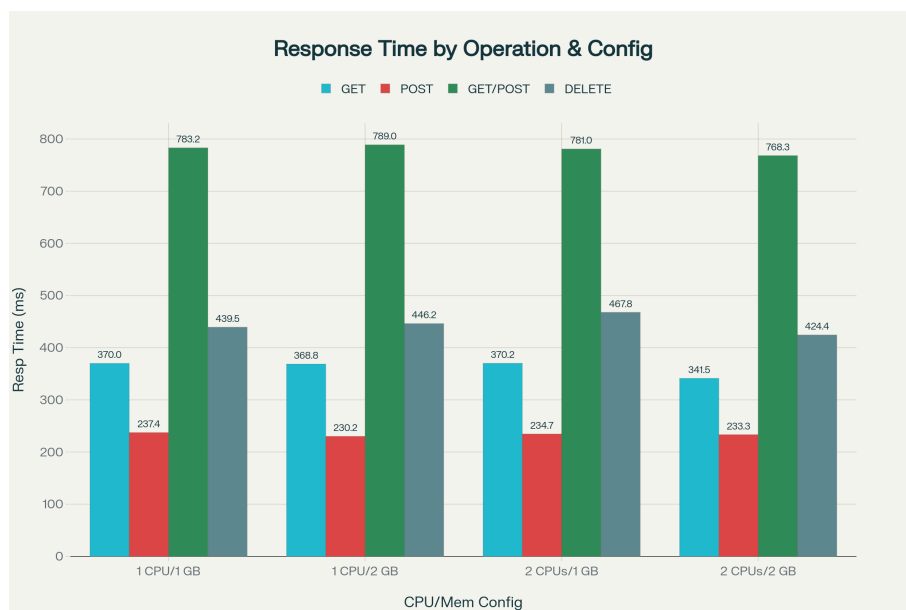


Figure 1. Observou-se que as requisições POST apresentaram os menores tempos de resposta em todas as configurações avaliadas, enquanto o cenário combinado GET/POST exibiu os maiores tempos, sugerindo sobrecarga adicional quando múltiplos endpoints são exercitados em conjunto. Os resultados para DELETE permaneceram em patamar intermediário, e incrementos de CPU/Memória alteraram pouco a ordem relativa entre operações nesse regime de carga.

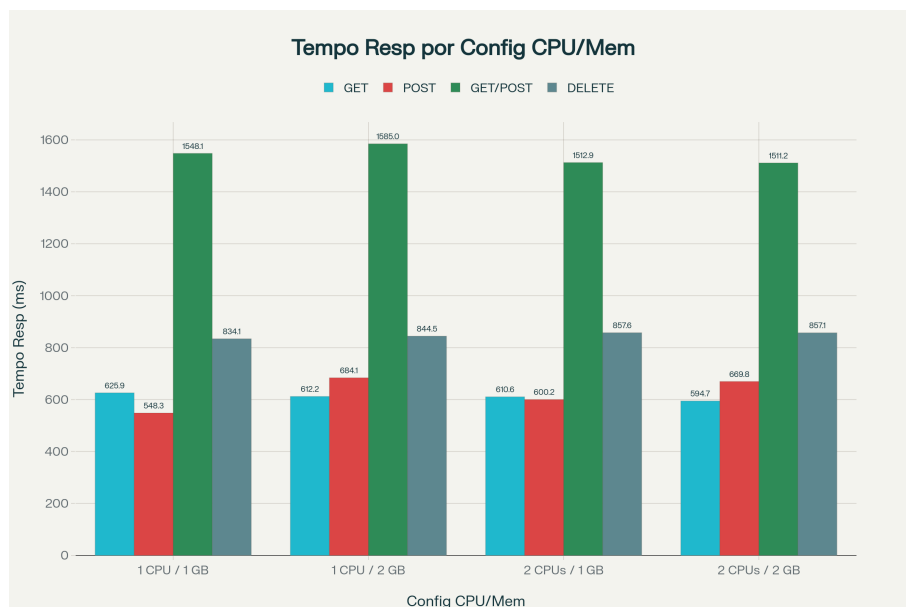


Figure 2. O perfil de desempenho manteve-se estável: o mix GET/POST continuou sendo o mais lento em todas as combinações de hardware, ao passo que GET e POST permaneceram abaixo da ordem de centenas de milissegundos, com DELETE na faixa próxima de 0,8–0,86 s. A ampliação de recursos (2 CPUs/2 GB) resultou em reduções modestas para GET, mas não mitigou substancialmente o custo do cenário combinado GET/POST.

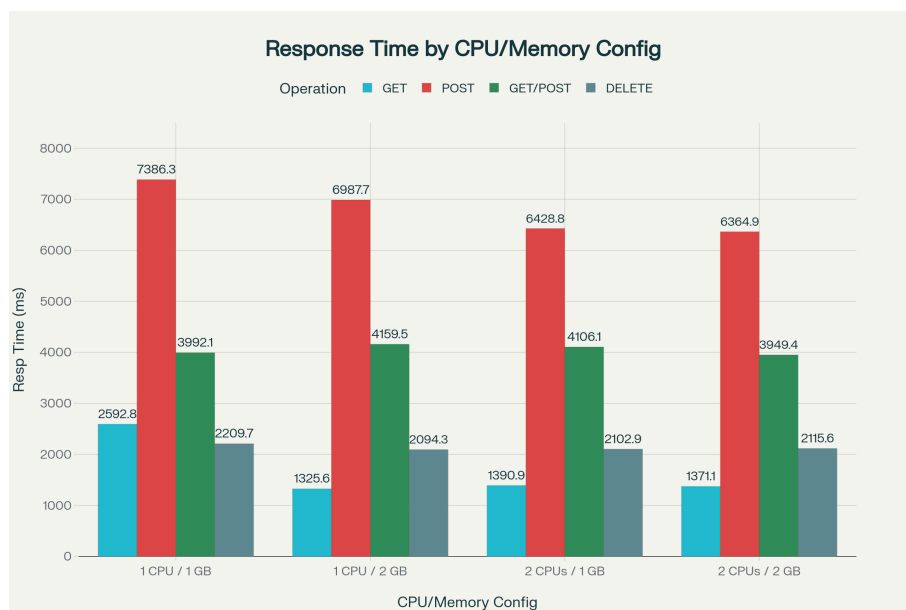


Figure 3. Verificou-se mudança de regime com degradação clara: POST passou a concentrar os maiores tempos de resposta em todas as configurações, seguido por GET/POST, enquanto GET e DELETE permaneceram relativamente menores. O aumento de CPU/RAM reduziu tempos para GET, porém teve impacto limitado em POST, indicando gargalos prováveis em camadas de escrita do backend ou do SGBD sob maior concorrência.

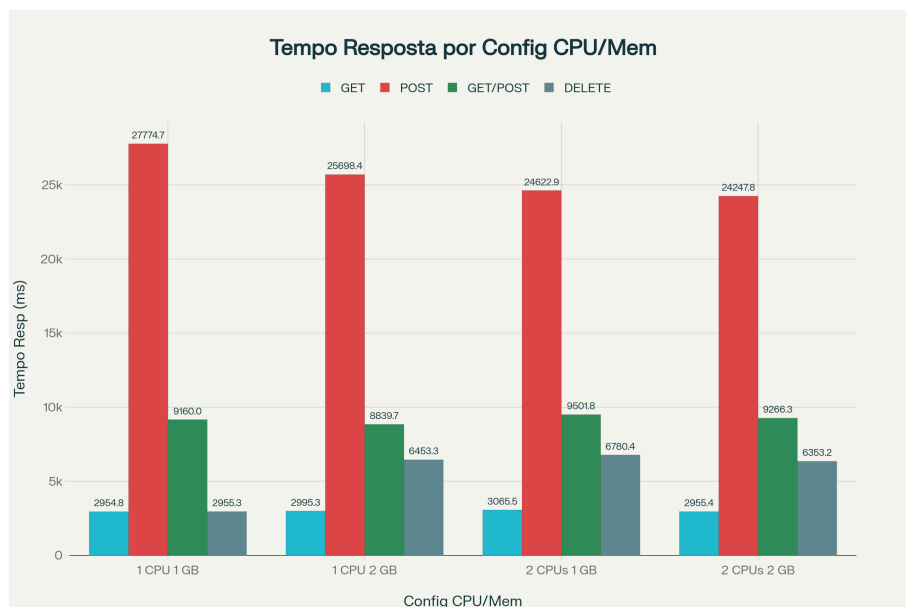


Figure 4. A degradação intensificou-se: tempos de POST cresceram uma ordem de magnitude, tornando-se o principal gargalo; GET/POST veio em seguida, enquanto GET e DELETE situaram-se na ordem de segundos (3–6,8 s) com variação sensível por hardware. Incrementos de CPUs/memória não se mostraram suficientes para conter os tempos de POST nesse patamar de carga, reforçando a hipótese de contenção na camada de escrita ou transacional do banco.

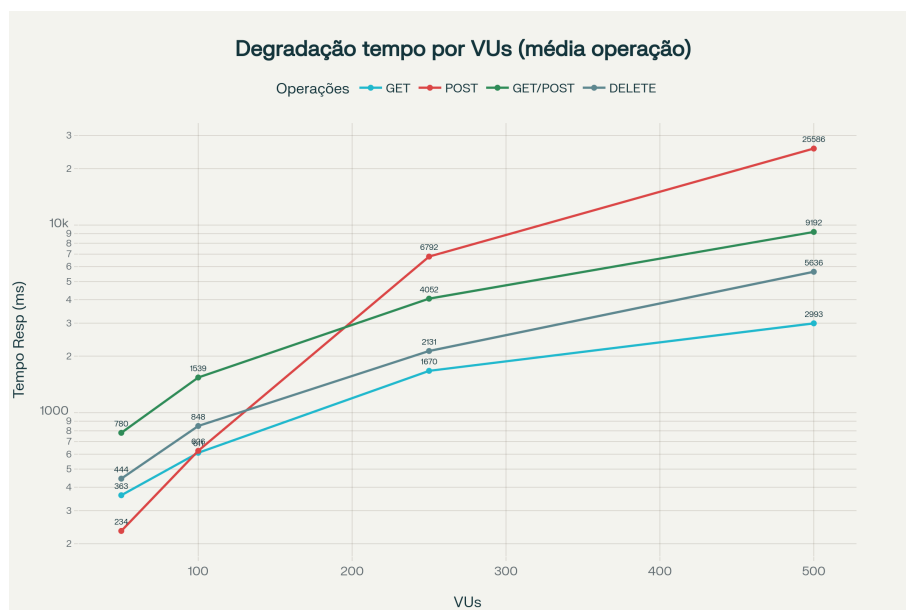


Figure 5. Em escala logarítmica, evidencia-se que as escalas de tempo mudam rapidamente já a partir de 100 usuários, com inflexões mais pronunciadas a partir de 250, sobretudo para POST e GET/POST, que migram de centenas de milissegundos para múltiplos segundos. GET e DELETE também sofrem elevação, embora menos acentuada, sinalizando limites de throughput e contenção sistêmica que se manifestam de forma não linear com o aumento de concorrência.

5. Conclusões e Trabalhos Futuros

Os resultados obtidos neste estudo permitiram compreender de forma abrangente o comportamento de aplicações Python/Django em ambientes containerizados sob diferentes condições de carga. Foi possível estruturar uma metodologia sistemática de avaliação de desempenho que alia controle de recursos computacionais, observabilidade contínua e aplicação de testes de carga progressivos, resultando em uma abordagem consistente e reproduzível para estudos experimentais com containers.

A arquitetura analisada apresentou estabilidade satisfatória em cenários de até 100 usuários simultâneos, mantendo baixos tempos de resposta e ausência de falhas relevantes. Contudo, observou-se degradação significativa do desempenho em cargas superiores, principalmente devido à saturação da CPU e ao aumento do custo das operações de escrita e exclusão de dados. A adoção do Gunicorn aliado ao Uvicorn mostrou-se vantajosa, proporcionando ganhos médios de cerca de 20% em desempenho em comparação ao servidor nativo do Django, o que reforça a importância do uso de servidores otimizados em ambientes de produção.

Do ponto de vista metodológico, destaca-se a utilização do Docker-Manager, uma aplicação desenvolvida no âmbito do projeto para orquestrar e automatizar a execução dos experimentos. Essa ferramenta possibilitou a replicação dos testes e a padronização da coleta de métricas, configurando-se como um avanço prático para estudos de desempenho e uma contribuição relevante para a criação de ambientes experimentais reproduzíveis em pesquisas com containerização.

Em perspectiva comparativa, os achados indicam que, embora o Python/Django

apresente limitações sob alta concorrência quando comparado a frameworks como Node.js e Java, sua combinação com Docker e servidores otimizados se mostra eficiente e adequada para aplicações de médio porte. Assim, este trabalho contribui tanto no âmbito técnico, ao identificar limites e boas práticas de uso, quanto no âmbito metodológico, ao empregar um arcabouço automatizado para análise de desempenho em sistemas containerizados.

Como continuidade, recomenda-se a investigação de estratégias de escalabilidade horizontal, com múltiplas réplicas do backend e balanceamento de carga, além da otimização transacional do PostgreSQL e da comparação entre diferentes plataformas de containerização. Essas linhas futuras podem aprofundar a compreensão sobre o comportamento e a eficiência de arquiteturas baseadas em Python/Django sob condições de alta demanda.

6. Referências

References

- Bueno, R. N. (2023). Desempenho das linguagens javascript e python para aplicações de alto desempenho em ambiente serverless. UNESP.
- Django Software Foundation (2025). Django documentation. <https://docs.djangoproject.com/en/5.2/>.
- Docker Inc. (2025). Official documentation. <https://docs.docker.com/>.
- Grafana Labs (2025). Grafana k6 documentation. <https://grafana.com/docs/k6/latest/>.
- Gunicorn Project (2025). Wsgi server documentation. <https://docs.gunicorn.org/en/stable/>.
- IEEE (2021). Containerization: Cloud computing based inspiration technology for adoption through docker and kubernetes. In *IEEE Xplore*.
- Iyer, G. R. (2015). Node.js: Event-driven concurrency for web applications. https://www.researchgate.net/publication/280546121_Nodejs_Event-driven_Concurrency_for_Web_Applications.
- Pandita, V. (2024). Benchmarking the performance of java virtual threads in high-throughput workloads. Master's thesis, National College of Ireland.
- Uvicorn Project (2025). Asgi server documentation. <https://uvicorn.dev/>.